

Документирование решений: Architecture Decision Records (ADR)

AI Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Фомин Дмитрий

Руководитель отдела архитектуры

- Больше 15 лет опыта работы архитектором
- Участие в разработке архитектуры в очень больших проектах (BSS)

 @advoc_diaboly

 fomin.dmitry@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **#OTUS**
AI-Architect-2026-04



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Стратегический фундамент
и планирование проекта

Проектирование и
документирование архитектуры

Продвинутые
архитектурные
паттерны

Инфраструктура

Качество, интеграции
и безопасность

Стратегия, лидерство и
экономика

Проектная работа

ФИНИШ



Маршрут вебинара

Знакомство

Введение и философия

ADL/ADR

Жизненный цикл и Git

Пример

Презентация решения



Цели вебинара

К концу занятия вы сможете

использовать формат ADR для фиксации
ключевых архитектурных решений, их
контекста и последствий;

формировать базу знаний по проекту,
которая упрощает онбординг новых
участников и эволюцию архитектуры.

Смысл

Для чего вам это уметь

для того, чтобы можно было
относительно спокойно сходить в отпуск

**Ведёте ли Вы реестр
архитектурных решений?**



Введение и философия

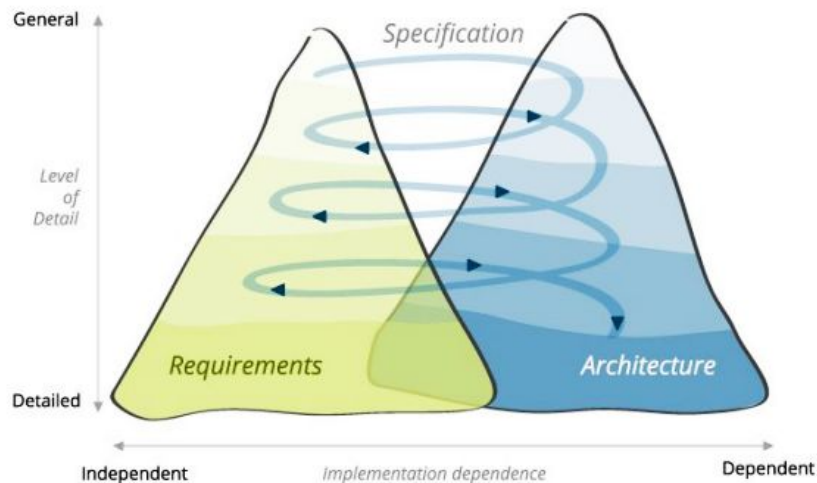
**В сущности, все модели
неправильны, но
некоторые полезны.**

Джордж Бокс, британский статистик

Главные законы архитектуры

Закон 1. «В архитектуре всё — **компромисс**»

Закон 2. «**Почему** в архитектуре важнее, чем **как**»



Артефакты архитектуры

Одна из задач архитектора – это передать архитектурные решения (видение решения и высокоуровневая спецификация).



Артефакты архитектуры

Архитектор помогает в превращении требований в:

- видение решения
- высокоуровневую спецификацию
- набор задач

Артефакты архитектуры

Описание решения (высокоуровневая спецификация) необходима для того, чтобы скоординировать взаимодействие нескольких людей или нескольких команд.

Описание решения необходимо для того, чтобы выразить все то, что плохо выражено или не может быть выражено в коде.

Артефакты архитектуры

Чем среди большего количества требуется координация, тем строже, формальнее и «бюрократичнее» документация и описание.



<http://agilemindset.ru/эффективные-способы-описания-архите/>

Специфика AI: Управление неопределенностью

IT до AI

Logic-driven.

Главный риск — Баги.

Документируем — Логику.

IT в эпоху AI

Data-driven.

Главный риск — Галлюцинации и Дрифт.

Документируем — Данные и Метрики.

Новые архитектурные драйверы (AI NFRs)

- В классическом ПО мы оптимизируем: Latency, Throughput, Availability.
- В AI появляются новые нефункциональные требования (NFR):
 - Explainability (Объяснимость): Можем ли мы объяснить регулятору, почему нейросеть отказала в кредите?
 - Fairness (Честность): Не дискриминирует ли модель по полу/возрасту?
 - Freshness (Свежесть знаний): Знает ли модель новости за вчера?
- Типичный конфликт: Нейросети точнее (High Accuracy), но их сложнее объяснить (Low Explainability). Деревья решений проще объяснить, но они менее точные.
- Одной из наших целей должно стать фиксирование выбора между противоречащими целями

Вопросы



если есть вопросы



если вопросов нет

ADL/ADR

Architecture Decision Log (ADL)– это

регулярная фиксация принятых и непринятых в ходе разработки программного обеспечения решений, затрагивающих дизайн, проектирование, выбор инструментов и подходов, и отвечающих определенным функциональным или нефункциональным требованиям.

Зачем нужен ADL?

- Повысить качество решения
- Ответить на вопрос, почему было принято такое решение

<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>



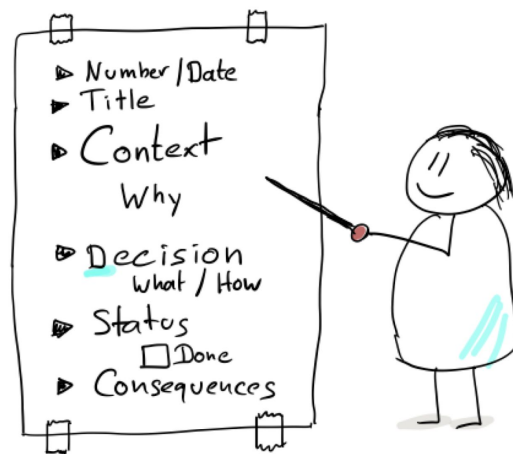
Architecture Decision Record (ADR)– это

единичная запись из ADL

ADR обычно включает в себя:

- Контекст проблемы, которую решаем
- Возможные альтернативы решения
- Решение
- Последствия решения
- Риски, сильные стороны, компромиссы и т.д.

<https://github.com/joelparkerhenderson/architecture-decision-record>



Пример ADR

Decision record template by Michael Nygard

This is the template in [Documenting architecture decisions - Michael Nygard](#). You can use [adr-tools](#) for managing the ADR files.

In each ADR file, write these sections:

Title

Status

What is the status, such as proposed, accepted, rejected, deprecated, superseded, etc.?

Context

What is the issue that we're seeing that is motivating this decision or change?

Decision

What is the change that we're proposing and/or doing?

Consequences

What becomes easier or more difficult to do because of this change?

Файл ADR обычно именуется с указанием id решения и краткого описания:

NNN-краткое-описание.md

Пример:

001-выбор-базы-данных.md

002-использование-REST-API.md

<https://github.com/joelparkerhenderson/architecture-decision-record/tree/main/locales/en/templates/decision-record-template-by-michael-nygard#decision-record-template-by-michael-nygard>



Контекст

- Это не просто описание задачи. Это ограничения.
- Пример: Не «Нам нужна база данных», а «Нам нужна БД с поддержкой ACID и гео-репликацией, так как мы выходим на рынок Азии, но бюджет ограничен X».

Альтернативы: ценность отвергнутых решений

- Что писать:
 - Перечислить 2-3 реальные опции (не выдуманные).
 - Кратко: «Хорошо, так как...» и «Плохо, так как...»
- Пример:
 - Опция 1: PostgreSQL. Хорошо: стандарт компании, ACID. Плохо: плохо масштабируется на запись.
 - Опция 2: Cassandra. Хорошо: отличная запись. Плохо: команда не имеет экспертизы, дорого поддерживать.

Решение

- Пишем в активном залоге: «Мы будем использовать Postgres 14».
- Это утверждение, а не рассуждение.

Последствия

- Любое решение имеет цену.
- Что стало лучше? (Плюсы).
- Что стало хуже/сложнее? (Минусы).
- Пример: «Внедрили Kubernetes -> Выросла сложность деплоя, нужен DevOps, но получили автомасштабирование».



Когда писать ADR?

Высокое влияние/Легко отменить

UI-компоненты

Решение: Обсуждаем, пишем гайд

Высокое влияние/Трудно отменить

Выбор СУБД, Языка, Фреймворка, LLM

Решение: ОБЯЗАТЕЛЬНО пишем ADR

Низкое влияние/Легко отменить

Именованье класса, пакета

Решение: Не обсуждаем, не пишем ADR.

Низкое влияние/Трудно отменить

Правила именования, версии, code-style

Решение: Обсуждаем, фиксируем как соглашение

Хороший vs Плохой ADR

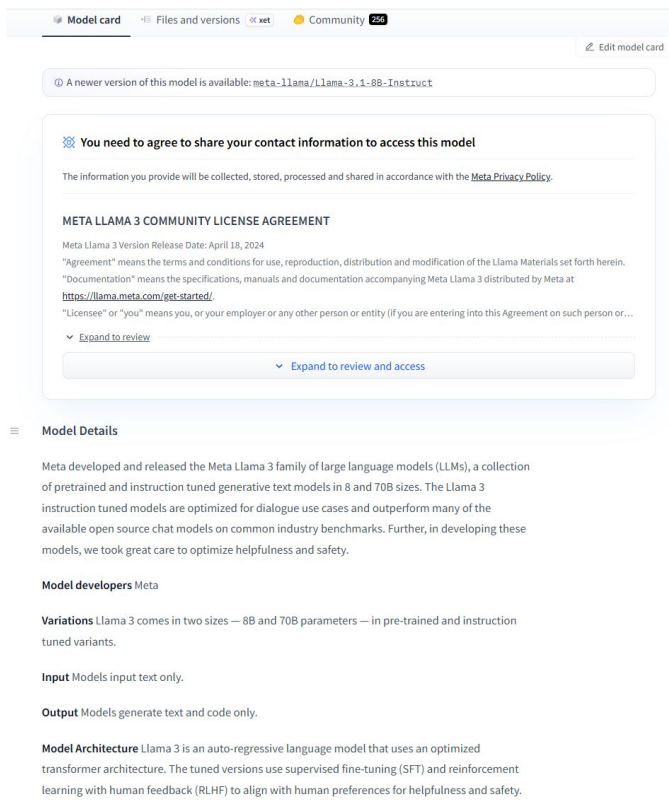
Выбрали React, так как команда знает JS, есть готовая библиотека компонентов X, но придется переписать легаси на jQuery

Выбрали React, потому что модно

AI специфика в стандартном ADR

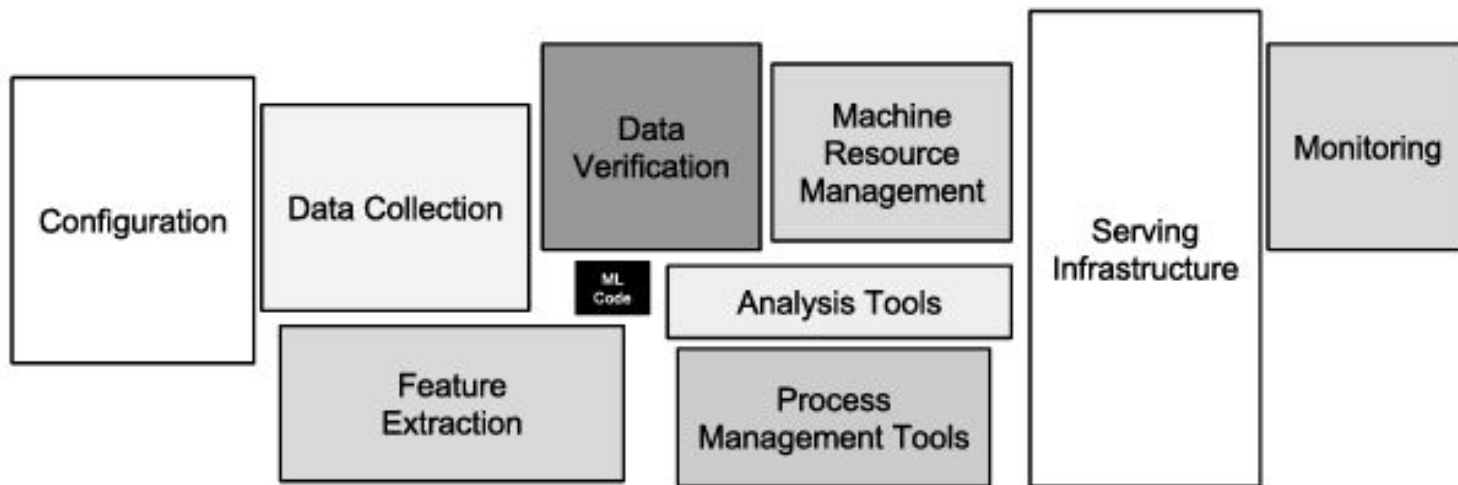
- В AI проектах имеет смысл расширить стандартный формат за счёт дополнительного раздела: Compliance & Ethics
- В нём имеет смысл описать специфические для AI риски, которые нельзя игнорировать при проектировании. Что пишем:
 - Риск галлюцинаций и фактологических ошибок.
 - Предвзятость (Bias) обучающей выборки.
 - Риск генерации токсичного контента (Jailbreak).
 - Соответствие регуляторике (152-ФЗ, GDPR, AI Act).

Model Card как разновидность ADR



- Если ADR описывает решение о выборе, то Model Card описывает саму модель.
- Зачем нужно: Чтобы DS, инженеры и бизнес понимали границы применимости модели (где она работает хорошо, а где её использовать опасно).
- Структура:
 - Model Details: Версия, дата, лицензия.
 - Intended Use: Для чего создавалась (например, саммаризация диалогов).
 - Out-of-score: Для чего НЕЛЬЗЯ использовать (например, медицинские диагнозы).
 - Training Data: На чем училась (копирайт, приватность).
 - Metrics: Результаты тестов (Accurasy, F1, Latency).

Скрытая сложность AI архитектур



<https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>

Вопросы



если есть вопросы



если вопросов нет

Жизненный цикл и Git

Architecture as Code (AaC)

- Принцип: Документация живет там же, где и код (в репозитории), и подчиняется тем же процессам.
- Почему Text-based форматы (Markdown, PlantUML) побеждают бинарные (Visio, Word):
 - Версионирование: Git хранит историю изменений каждой строки.
 - Diff & Merge: Можно легко сравнить две версии документа и увидеть, что именно изменилось (в отличие от картинки PNG/VSD).
- Single Source of Truth: Код и описание архитектуры синхронизированы через единый релизный цикл.

Инструменты визуализации

Диаграммы

- <https://www.archimatetool.com/>
- <https://miro.com/>
- <https://app.diagrams.net/>

АоС

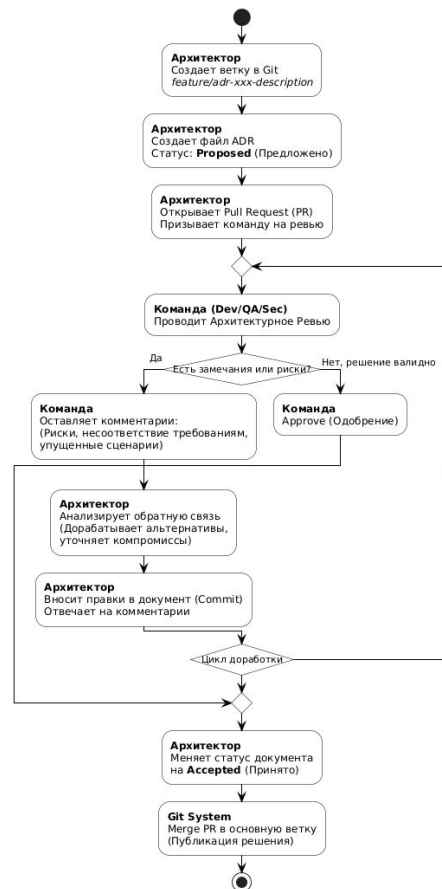
- <https://plantuml.com/>
- <https://mermaid-js.github.io/>
- <https://structurizr.com/>

Инструменты автоматизации

- Создание и управление (CLI):
 - adr-tools: Стандарт де-факто (Bash-скрипты).
 - Команда `adr new "Use PostgreSQL"` автоматически:
 - Создает файл с правильным номером (напр., `005-use-postgresql.md`).
 - Проставляет дату и статус `Proposed`.
 - Обновляет оглавление (`README.md`).
- Публикация (Rendering):
 - Markdown хорош для разработчиков, но плох для менеджеров.
 - Решение: Генераторы статических сайтов (MkDocs, Hugo, Docusaurus).
 - Автоматическая сборка документации в красивый HTML-сайт при каждом коммите (CI/CD).
- Визуализация истории:
 - Инструменты вроде log4brains позволяют строить граф зависимостей решений и timeline проекта.

Процесс работы

- Проблема: Возникает архитектурный вопрос (например, «Какую БД выбрать?»).
- Процесс
 - Ветка: Архитектор создает ветку feature/adr-005-database.
 - Draft: Создает файл docs/adr/005-postgres.md со статусом Proposed.
 - Pull Request: Открывает PR. Призывает лидов разработки и QA.
 - Review:
 - Dev: «Postgres не вывезет нашу нагрузку на запись!»
 - Arch: «Ок, добавим шардирование в раздел Consequences или сменим на Cassandra».
 - Merge: После апрува статус меняется на Accepted, PR вливается в main.



Жизненный цикл статусов

- Активные статусы:
 - Proposed (Предложено): Черновик, отправлен на ревью (Pull Request).
 - Accepted (Принято): Одобрено командой, решение внедряется.
- Финализирующие статусы:
 - Rejected (Отклонено): Решение рассмотрели, но отказались (важно сохранить причину отказа!).
- Эволюционные статусы (со временем):
 - Deprecated (Устарело): Решение было правильным раньше, но сейчас мешает (Технический долг).
 - Superseded (Заменено): Решение заменено новым ADR (ссылка: Superseded by ADR-010).
- ADR никогда не удаляются. Мы храним историю ошибок и изменений.



Альтернативный пример лога решений

DECISION LOG					
ID	CATEGORY	DECISION	RESPONSIBLE PARTY	DECISION DATE	COMMENTS
D001	Approval	CRQ015-WP1 approval agreed at the 15 October Joint Project Board. Follow-on action on S. Dole to arrange signatures, and complete project documentation updates.	Joint Project Board	15-Oct	Change is required to fix defect SD0041.
D002	Process	Decision made by S. Dole and F. Smith on daily stand up to complete 'nice to have' defects after go live.	S. Dole and F.Smith	10-Nov	See defect log in sharepoint.
D003	Technical	Dev team have confirmed decision to use jquery plugin for location finder.	A. Peer	10-Nov	Alternatives considered are documented in the Tech Design.
D004	Technical	Sec Ops have reviewed pen test results and agreed to go ahead with implementation of JAX SaaS tool. Vulnerabilities are minor and will be removed with upgrade scheduled for Jan.	D. Orey	13-Oct	See notes of sec ops meeting 10 October.

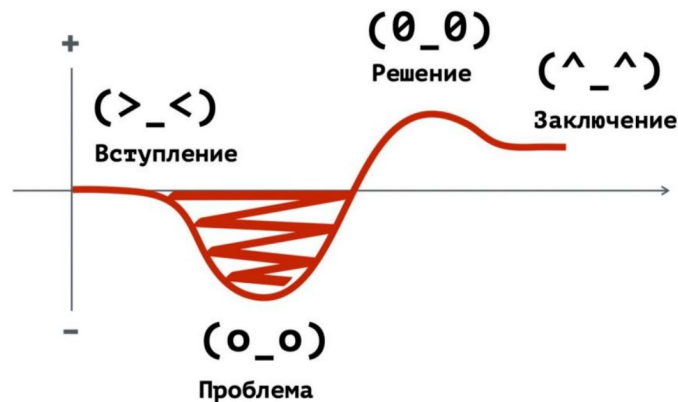
Топология ADR: От микросервиса до предприятия

- Уровень Сервиса
 - Решения: "Выбор библиотеки JWT"
 - Репозиторий auth-service -> папка /docs/adr.
- Уровень Системы (Solution)
 - Решения: "Схема взаимодействия сервисов", "Выбор Event Bus"
 - Отдельный репозиторий banking-system-arch
- Уровень Компании (Enterprise)
 - Решения: "Запрет на использование MongoDB", "Стандарт логирования"
 - Репозиторий tech-standards

Презентация решения

Защита решения: Сторителлинг

- ADR — это книга, Презентация — это спектакль.
- Люди не запоминают списки фактов, люди запоминают истории.
- Структура питча:
 - Экспозиция (Вступление): «Мы жили спокойно, клиенты звонили...»
 - Инцидент (Проблема/Контекст): «...но время ожидания выросло до 10 минут. Бизнес теряет деньги.»
 - Кризис (Альтернативы): «OpenAI нам нельзя (тюрьма), писать самим — долго.»
 - Кульминация (Решение): «Мы нашли Llama-3 + Квантование.»
 - Развязка (Заключение): «Да, качество чуть хуже, зато безопасно и бесплатно.»



Вопросы



если есть вопросы



если вопросов нет

Пример

Уточняем задачу про чат-бот

- Продукт: Внутренний чат-бот для операторов колл-центра **банка**. Поиск по базе знаний **инструкций ЦБ и регламентам**.
- Текущая проблема: Операторы тратят 5+ минут на поиск ответа. Клиенты висят на линии.
- Бизнес-цель: Сократить время поиска до 30 секунд.
- Жесткие ограничения (Constraints):
 - 152-ФЗ / Банковская тайна: Данные клиентов и внутренние регламенты ни в коем случае не должны покидать контур банка.
 - Бюджет: Выделено 500 тыс. руб. на стартовую инфраструктуру (нет бюджета на кластер H100).
 - стек команды: Python (Middle), опыт в ML — начальный.

Анализ Альтернатив

Критерий	Вариант А: SaaS (OpenAI/Azure)	Вариант Б: Self-Hosted (Llama 3 / Mistral)
Качество ответов	★★★★★	★★★★ (Хорошо, но возможны галлюцинации)
Безопасность (DLP)	❌ Риск (Данные потенциально уходят провайдеру)	✅ Полный контроль (Внутри периметра)
Time-to-Market	🚀 1 неделя (Только API)	🕒 1-2 месяца (Закупка железа, настройка)
Стоимость (Cost)	OpEx (Платим за токены)	CapEx (Покупка GPU)
Сложность поддержки	Низкая	Высокая (Нужен MLOps)

Внутренний Чат-бот. Дополнительные требования

Тип	Требование / Ограничение	Архитектурное решение
Security	Данные клиентов и внутренние регламенты ни в коем случае не должны покидать контур банка.	Будем использовать Self-hosted модель
Budget	Нет бюджета на кластер H100	Будем использовать модель Llama-3-8B-Instruct с квантованием 4-bit.



ADR-005: Использование локальной LLM (Llama 3)

```
# ADR-005: Выбор движка LLM для ассистента поддержки
```

```
**Status:** Accepted
```

```
**Date:** 2024-05-20
```

```
## Context
```

Нам необходимо внедрить семантический поиск по базе знаний.

Служба ИБ (Пр. №12) запрещает передачу текстов регламентов в облачные API.

Бюджет на GPU ограничен одной картой RTX 4090.

```
## Alternatives
```

Мы рассмотрели два основных подхода к реализации:

```
1. **Azure OpenAI (SaaS, GPT-4)**
```

- * **Плюсы:** Мгновенный старт (Time-to-Market < 1 недели), эталонное качество генерации.

- * **Минусы:** Данные покидают периметр банка. Категорический запрет от службы ИБ.

- * **Статус:** **Rejected** (Blocker по безопасности).

```
2. **Self-hosted Llama-3-8B (On-premise)**
```

- * **Плюсы:** Данные не покидают закрытый контур. Полный контроль над инференсом. Отсутствие лицензионных платежей (Community License).

- * **Минусы:** Требуется закупка оборудования (GPU). Требуется компетенций DevOps/MLOps для настройки и квантования. Качество ниже, чем у GPT-4.

ADR-005: Использование локальной LLM (Llama 3)

Decision (Решение)

Мы будем использовать ****Self-hosted модель Llama-3-8B-Instruct**** с квантованием 4-bit.

Запуск через vLLM сервер внутри контура банка.

Consequences (Последствия)

****Положительные:****

+ Полное соответствие 152-ФЗ.

+ Отсутствие переменных расходов (OpEx) на токены.

****Отрицательные (Trade-offs):****

- Качество ответов ниже, чем у GPT-4 (риск неверных ответов по сложным запросам).

- Требуется выделение ресурса DevOps на настройку мониторинга GPU.

- Скорость генерации ограничена ~50 токенов/сек (достаточно для текста, но медленно для больших отчетов).

Compliance & Ethics

- Используется базовая Instruction-tuned версия. Риск предвзятости существует.

Как будем снижать риски:

Внедрить пост-фильтрацию ответов (Guardrails) перед отдачей пользователю для отсека токсичности. Добавить дисклеймер: "Я бот, могу ошибаться".

Как делать НЕ надо (Анти-паттерн)

```
# ADR-012: LLM
```

```
Status: Done
```

Контекст:

Нужен AI.

Решение:

Берем Llama 3, потому что это Open Source и бесплатно.

И еще она популярна на GitHub.

Последствия:

Все будет хорошо.



Вопросы



если есть вопросы



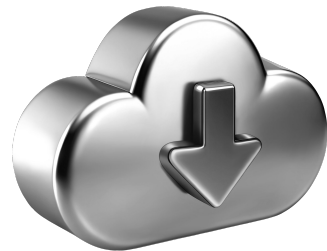
если вопросов нет



Список материалов для изучения

Примечание: Список для этого урока будет короче, так как основные источники уже были использованы в других уроках.

1. Pahl, Claus, et al. 2023. Software Architecture: A Comprehensive Framework and Guide for Practitioners. Berlin: Springer.



**Подведём
итоги занятия**

Теперь вы сможете:

Отправьте в чат номера достигнутых целей

использовать формат ADR для фиксации
ключевых архитектурных решений, их
контекста и последствий;

формировать базу знаний по проекту,
которая упрощает онбординг новых
участников и эволюцию архитектуры.

для того, чтобы можно было
относительно спокойно сходить в отпуск

Вопросы для проверки

1. Если решение устарело и мы переписали код, файл старого ADR нужно удалить, чтобы не засорять репозиторий?
2. В разделе "Последствия" (Consequences) нужно перечислять только выгоды, которые получит бизнес?
3. ADR пишет только архитектор в одиночку, а команда просто читает?

Ключевые тезисы занятия

1. «Почему» важнее, чем «Как».
2. Нет «Последствий» = Нет Архитектуры.
3. Docs as Code спасает от устаревания



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

